INTERFACE SPECIFICATION · SAP R/3 ↔ PO GOVERNANCE · V1.4

SAP R/3 integration API: structure and contract

The exact API the SAP team builds against to link R/3 with the PO Governance server: endpoints, fields, security, error and retry rules, and how SAP should call them. A machine-readable copy (OpenAPI 3.0) accompanies this document.

Companion file: po-governance-sap-api.yaml Audience: SAP ABAP, workflow and Basis teams; IT network/security

1 Oct 2026 · v1.4

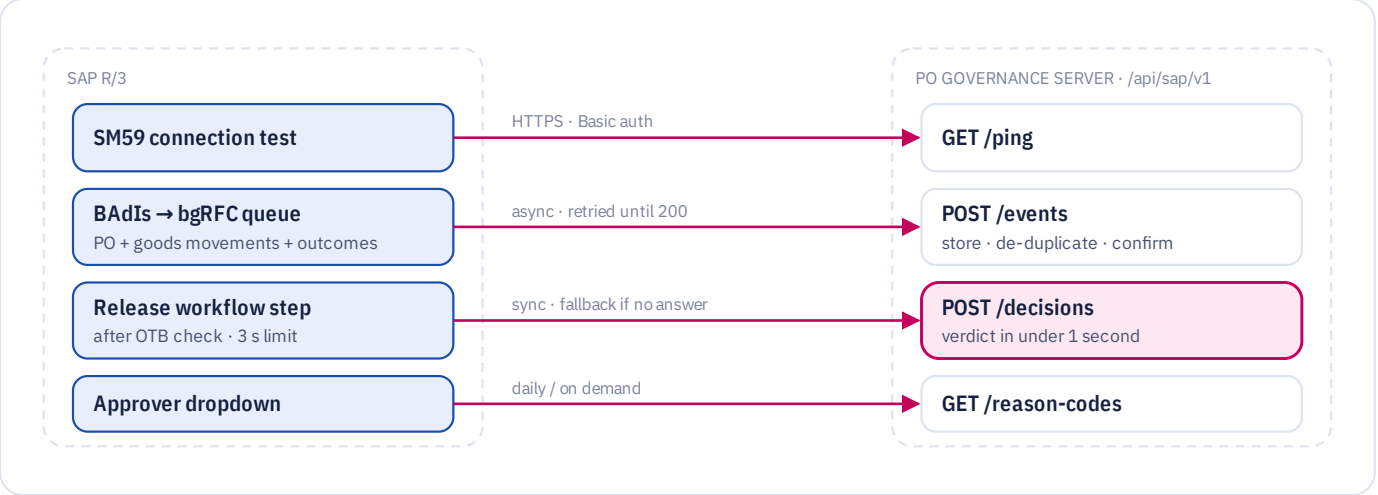
CONTENTS

1. Overview
2. Connection and security
3. Common rules
4. GET /ping
5. POST /events
6. POST /decisions
7. Governance outcomes
8. GET /reason-codes
9. Errors and retries
10. SAP implementation notes
11. Test checklist
12. Tool-side build status

01 Overview

IN SIMPLE WORDS

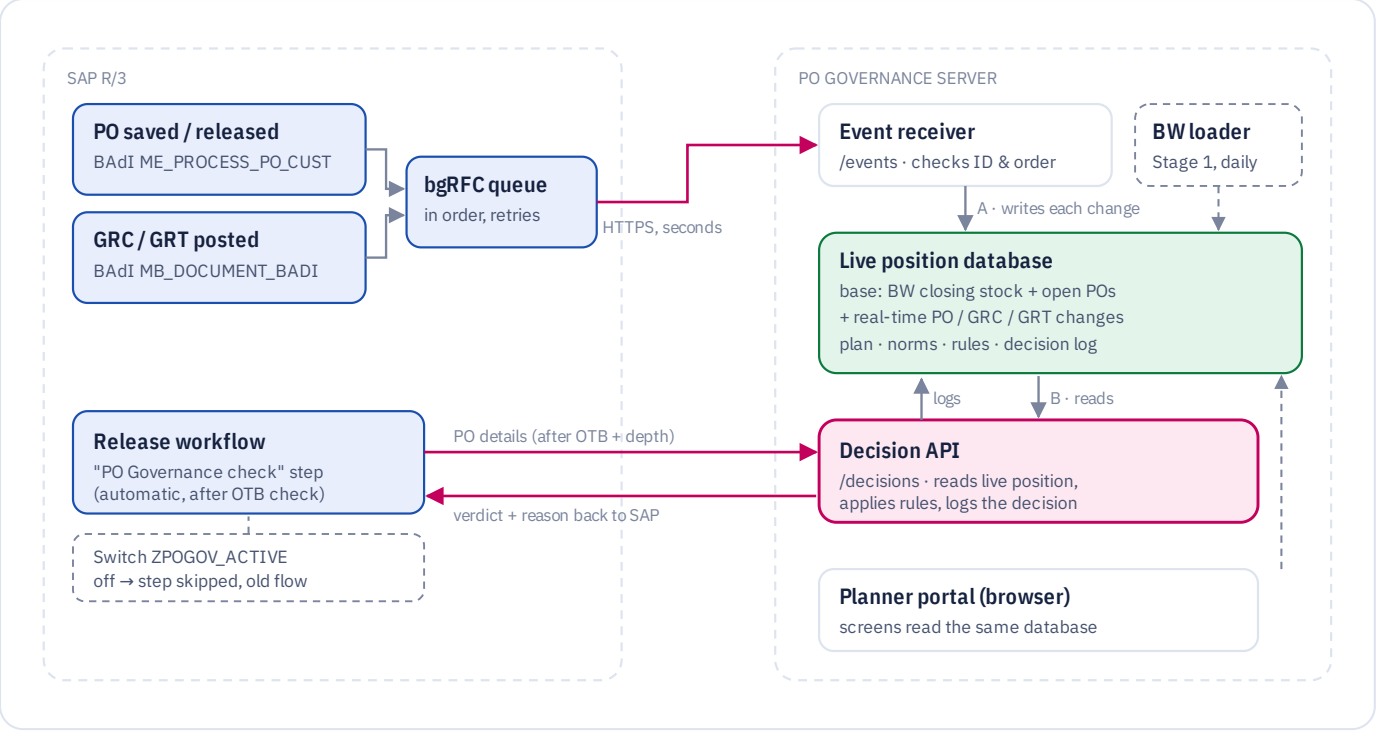
The link has only **four "doors"** on our server, and SAP always knocks; our server never logs in to SAP. **/ping** is a doorbell to test the line. **/events** is the letterbox where SAP drops a slip every time a PO or goods movement is saved. **/decisions** is the checker's window: SAP hands over a PO with its latest figures and gets the verdict back within a second. **/reason-codes** gives SAP the list of exception reasons to show approvers.



ENDPOINT	CALLED BY	STYLE	PURPOSE
`GET /ping`	Basis (SM59 test), monitoring	Sync	Check the connection and credentials
`POST /events` **Stage 2**	bgRFC unit, after commit	Async, guaranteed	PO create/change/release/close, goods receipts and returns (vendor POs), governance outcomes
`POST /decisions` **Stage 2**	Workflow background step	Sync, ≤ 3 s	Verdict (Allow / Reduce / Watch / Hold) for a PO, using the tool's live position
`GET /reason-codes`	SAP program (daily) or approver screen	Sync	List of valid exception reason codes

Inside the server: how /events and /decisions stay in sync

The event receiver (`/events`) and the Decision API (`/decisions`) are part of the same server program and share **one live-position database**. Nothing is copied between them. The receiver applies each event in a single database transaction and confirms to SAP only after the commit, so every confirmed event is visible to the next decision. The Decision API reads the position in one consistent read, leaves out the PO being checked, checks that the feed is not delayed, and logs the decision with the last event sequence applied.



Stage 1 (BW files) fills the same database each morning: the new base is loaded and the previous day's adjustments are cleared in one transaction. **Stage 3** (AI layer) needs no new SAP interface; it uses the decision log and the `GOV_OUTCOME` events defined in this specification.

02 Connection and security

ITEM	VALUE
Base URL (production)	`https://<vm-dns-name>:8443/api/sap/v1`
Base URL (test)	`https://<test-vm-dns-name>:8443/api/sap/v1`, connected to SAP QA only
Transport	HTTPS (TLS 1.2+), certificate issued by the company CA; SAP imports the CA chain in **STRUST** (SSL client PSE)
Authentication	HTTP Basic, one technical account per SAP system (`sap_ecq`, `sap_ecp`), password at least 24 characters, rotated yearly
Network	Firewall: SAP application servers → VM, TCP 8443. The tool also allow-lists those server IPs. **No connection from the VM into SAP is needed.**
Content type	`application/json; charset=utf-8` for requests and responses
Versioning	Version in the path (`/v1`). Within v1 only additive changes (new optional fields); SAP must ignore unknown fields in responses.

SM59 destination (type G, HTTP connection to external server)

SM59 FIELD	SETTING
RFC destination	`ZPOGOV_API` (and `ZPOGOV_API_TEST` in QA)

SM59 FIELD	SETTING
Target host / Service no.	VM DNS name / `8443`
Path prefix	`/api/sap/v1`
Logon & Security	Basic authentication with the technical account; SSL **Active**, SSL certificate = the client PSE holding the company CA
Special options	Timeout: 10 s for events; the workflow call sets its own 3 s timeout in code (section 10)
Connection test	Must return HTTP 200 from `/ping`

03 Common rules for every call

- **Field names** in `snake_case`. This matches ABAP structure names when serialised with `/ui2/cl_json` in `pretty_mode-low_case`.
- **Timestamps** ISO 8601 with offset, e.g. `2026-10-01T11:42:07+05:30`. **Dates** `YYYY-MM-DD`.
- **Quantities** as JSON numbers in the base unit of measure (normally PC), always positive; the direction of a goods movement comes from `debit_credit` (S/H). **Amounts** in document currency.
- **Article numbers** in SAP internal format, with leading zeros kept. The tool normalises them.
- **Empty values:** send `null` or omit the field. Never send placeholder text.
- **OTB** is never sent and never evaluated by the tool. It stays in SAP.
- Every response includes a `trace_id` on errors. Quote it when raising an issue.

04 Connection test

GET `/ping` No body. Returns 200 if reachable and authorised.

```
{ "status": "ok", "server_time": "2026-10-01T14:30:00+05:30", "api_version": "1.0.0",
  "environment": "PRODUCTION", "source_system_recognised": "ECP" }
```

05 Event push (Stage 2)

POST `/events` Batch of 1–500 events, sent from an ordered bgRFC queue after commit.

When SAP sends an event

BUSINESS ACTION IN SAP	EVENT_TYPE	HOOK (TYPICAL)	PAYLOAD
PO created / changed	`PO_CREATED`, `PO_CHANGED`	`ME_PROCESS_PO_CUST` → POST	purchase_order (full state)
PO released / release revoked / refused	`PO_RELEASED`, `PO_RELEASE_REVOKED`	Release BAdI / workflow event	purchase_order (full state)
PO closed (delivery completed) / deleted	`PO_CLOSED`, `PO_DELETED`	`ME_PROCESS_PO_CUST`	purchase_order (full state)
Goods receipt / reversal (101 / 102)	`GR_POSTED`, `GR_REVERSED`	`MB_DOCUMENT_BADI` → MB_DOCUMENT_UPDATE	goods_movement (one per line)
Return to vendor / reversal (122 / 161)	`RETURN_POSTED`, `RETURN_REVERSED`	`MB_DOCUMENT_BADI`	goods_movement
Buyer accepts / exception approved or rejected	`GOV_OUTCOME`	Workflow step	governance_outcome (section 07)

Delivery contract

- **HTTP 200** = all events in the batch are safely stored (accepted, duplicate or parked). SAP removes the unit from the queue.
- **Anything else, or no answer within 10 s** = SAP retries **the same batch** (bgRFC retry). Resending is always safe.
- **De-duplication** by `msg_id`. A repeated message is reported in `duplicates` and ignored.
- **Order** by `seq`, ascending per queue. Use one queue (e.g. `ZPOGOV_EVENTS`) so order is strict.
- **A bad event never blocks the queue:** it is *parked* on the tool side, listed in `parked`, and alerted to IT.
- **PO events carry the full current state** of the PO, not just what changed. The tool is self-correcting even if an event was ever missed.
- **Cut-off rule (link with Stage 1):** the day's base is BW's closing-stock snapshot, whose control file states the exact cut-off time. The tool applies only events with `occurred_at` after the cut-off. Each morning the previous day's adjustments are cleared (they are inside the new snapshot).
- **Scope:** vendor POs and movements against vendor POs only. Internal stock transfers (STO) are not sent: the tool works at company level, where transfers do not change

stock.

- **PO, GRC and GRT always travel in the same feed:** a GRC moves quantity from open PO to stock, so both must be updated at the same moment.

Batch envelope

FIELD	TYPE	DESCRIPTION
`source_system` *	string	SAP system ID (SY-SYSID), e.g. ECP
`client` *	string	SAP client (SY-MANDT)
`queue`	string	bgRFC queue name
`sent_at`	date-time	When the batch was sent
`events` *	array (1–500)	Events in order
`events[].msg_id` *	string ≤ 80	Unique ID built by SAP, e.g. `ECP300-GR-5000123456-2026-0001`
`events[].seq` *	integer	Ascending number per queue
`events[].event_type` *	enum	See table above
`events[].occurred_at` *	date-time	When the document was saved
`events[].sap_user`	string	SY-UNAME

purchase_order (full state after save)

FIELD	SAP SOURCE	DESCRIPTION
`po_number` *	EKKO-EBELN	PO number
`doc_type`	EKKO-BSART	PO type
`vendor`	EKKO-LIFNR	Vendor
`purch_org / purch_group`	EKKO-EKORG / EKGRP	Purchasing org / buyer group
`created_on / created_by`	EKKO-AEDAT / ERNAM	Creation date and user
`release_status`	EKKO-FRGKE / FRGZU	NOT_RELEASED · IN_RELEASE · RELEASED · REFUSED
`release_indicator`	EKKO-FRGKE	Raw indicator
`currency`	EKKO-WAERS	Document currency
`deleted`	—	true if the whole PO is cancelled
`items[].po_item` *	EKPO-EBELP	Item number
`items[].article` *	EKPO-MATNR	Variant / single article
`items[].generic_article`	MARA-SATNR	Generic article
`items[].colour`	Variant characteristic	Colour value

FIELD	SAP SOURCE	DESCRIPTION
items[].merch_category	EKPO-MATKL	MC
items[].site	EKPO-WERKS	Informational; the tool aggregates to company level
items[].order_qty * / uom *	EKPO-MENGE / MEINS	Ordered quantity, unit
items[].received_qty / returned_qty	EKBE	Totals so far
items[].open_qty	derived	order – received + returned; 0 if delivery completed or deleted
items[].net_price / price_unit	EKPO-NETPR / PEINH	Price
items[].delivery_date	EKET-EINDT	First schedule line date
items[].deletion_flag / delivery_completed	EKPO-LOEKZ / ELIKZ	Flags

goods_movement (one material-document line)

FIELD	SAP SOURCE	DESCRIPTION
material_doc / doc_year / doc_item *	MBLNR / MJAHR / ZEILE	Material document key
movement_type *	BWART	101, 102, 122, 161, 351, 641 …
debit_credit	SHKZG	S = in, H = out
posting_date	BUDAT	Posting date
article * / generic_article / colour	MATNR / SATNR / characteristic	Article
site / storage_location	WERKS / LGORT	Informational; company level is used
quantity * / uom *	MENGE / MEINS	Positive quantity, base unit
po_number / po_item	EBELN / EBELP	If the movement refers to a PO
reversal_of	SMBLN	Original document for reversals

Example request and response

```
POST /api/sap/v1/events
{ "source_system": "ECP", "client": "300", "queue": "ZPOGOV_EVENTS",
  "sent_at": "2026-10-01T11:42:08+05:30",
  "events": [ {
    "msg_id": "ECP300-GR-5000123456-2026-0001", "seq": 45121,
    "event_type": "GR_POSTED", "occurred_at": "2026-10-01T11:42:07+05:30", "sap_user": "DCUSER01",
    "goods_movement": { "material_doc": "5000123456", "doc_year": 2026, "doc_item": 1,
      "movement_type": "101", "debit_credit": "S", "posting_date": "2026-10-01",
      "article": "000000000100234567", "generic_article": "000000000100234500", "colour": "NAVY",
      "site": "DC01", "storage_location": "0001", "quantity": 240, "uom": "PC",
      "po_number": "4500098765", "po_item": 10, "reversal_of": null } } ] }

200 OK
{ "received_at": "2026-10-01T11:42:08+05:30",
  "accepted": ["ECP300-GR-5000123456-2026-0001"], "duplicates": [], "parked": [] }
```

06 Governance decision (Stage 2)

`POST` `/decisions`

Called by the workflow step "PO Governance check", after SAP's OTB and depth plan checks have passed.

- **SAP sends only the PO.** No stock or sales figures: the tool uses its own live position (BW closing stock + real-time PO / GRC / GRT events). If the event feed is behind the agreed limit, the response carries `data_delayed = true` and SAP continues the PO as today.
- **Depth plan bypass:** for a REDUCE, SAP stores `decision_id` and `max_qty` on the PO item. When the quantity is reduced to `max_qty` or less, SAP skips its depth plan check for that item and resubmits with `depth_check = BYPASSED` and `previous_decision_id`.
- **Timing:** the tool answers within 1 s (target). SAP waits at most **3 s**. On timeout, network error, 5xx or 422, SAP continues exactly as today and records `NOT_CHECKED`.
- **Idempotent:** the same `request_id` with the same body returns the same decision. Suggested format: work-item ID + PO + version.
- **Mode:** `SHADOW` = decide and record only; SAP must not route on the verdict. `LIVE` = SAP routes on `route_to`. The mode comes from the SAP switch `ZPOGOV_ACTIVE` (per division).

Request

FIELD	TYPE	DESCRIPTION
`request_id` *	string ≤ 80	Idempotency key
`mode` *	SHADOW | LIVE	From the SAP switch
`source_system` * / `client`	string	SY-SYSID / SY-MANDT
`otb_check` *	PASSED	Assertion that SAP's OTB check has passed
`depth_check` *	PASSED | BYPASSED	SAP's depth plan check result; BYPASSED only for a quantity reduced under a previous REDUCE
`previous_decision_id`	string	On resubmission after a REDUCE: the decision it was reduced under
`requested_at`	date-time	
`purchase_order.po_number` *	string	EBELN
`purchase_order.buyer_user` * / `buyer_email`	string	Buyer (for scope and audit)
`purchase_order.division`	string	For per-division rules and routing
`purchase_order.doc_type` / `vendor` / `purch_org` / `purch_group` / `currency`	string	As in events

FIELD	TYPE	DESCRIPTION
items[].po_item *, article *, generic_article, colour, merch_category, season, site		Item identification
items[].order_qty * / uom * / net_value / delivery_date		What is being ordered

Response (200)

FIELD	TYPE	HOW SAP USES IT
decision_id	string	Store in the workflow container / PO for audit
overall_verdict / verdict_code	ALLOW|REDUCE|WATCH|HOLD / A|R|W|H	Most severe item verdict; one-character code for SAP domain values
route_to	NONE|BUYER|CATHEAD|BMHEAD	Routing in LIVE mode (ignored in SHADOW)
bm_head_required	boolean	Escalate to B&M head as well (the approver itself is determined by SAP's release workflow)
summary_text	string ≤ 255	Show in the work item and the PO header note
items[].verdict / verdict_code		Per item
items[].max_qty	number	Maximum allowed qty when REDUCE
items[].reason_code / reason_text (≤ 132) / rule_id		Show to buyer and approvers
items[].metrics	object	plan, stock, open, headroom, cover days, sell-through (for display)
data_delayed	boolean	true → SAP continues the PO as today (NOT_CHECKED)
rules_version / data_as_of / decided_at		Audit

Routing in LIVE mode

OVERALL_VERDICT	ROUTE_TO	SAP WORKFLOW ACTION
ALLOW	NONE	Continue to hierarchy approvals; show "checked, OK"
REDUCE	BUYER	Return to buyer with max_qty and reason. Buyer changes qty (new check runs on resubmit) or requests an exception
WATCH	CATHEAD	Work item to the category head (as set in SAP's release workflow): approve with reason code, or reject
HOLD	CATHEAD or BMHEAD	As WATCH; B&M head as well when bm_head_required = true

Example

```
POST /api/sap/v1/decisions
{ "request_id": "WI000004711-4500099001-V1", "mode": "SHADOW", "source_system": "ECP", "client": "300",
  "otb_check": "PASSED", "depth_check": "PASSED", "requested_at": "2026-10-01T11:50:00+05:30",
  "purchase_order": { "po_number": "4500099001", "doc_type": "NB", "vendor": "0000201345",
    "purch_org": "1000", "purch_group": "B07", "buyer_user": "BUYER07", "division": "MENS", "currency":
"INR",
    "items": [ { "po_item": 10, "article": "000000000100234567", "generic_article": "000000000100234500",
      "colour": "NAVY", "merch_category": "MC1021", "season": "AW26",
      "order_qty": 1200, "uom": "PC", "net_value": 418800.00, "delivery_date": "2026-10-20" } ] } }

200 OK
{ "decision_id": "D-20261001-000457", "request_id": "WI000004711-4500099001-V1", "mode": "SHADOW",
  "overall_verdict": "REDUCE", "verdict_code": "R", "route_to": "BUYER",
  "bm_head_required": false,
  "summary_text": "Item 10: reduce to 500 - stock + pending already cover 75% of next month's plan",
  "rules_version": "R-2026-10-01", "data_delayed": false,
  "data_as_of": { "bw_snapshot": "2026-09-30T23:59:00+05:30", "events_lag_seconds": 2 },
  "decided_at": "2026-10-01T11:50:00+05:30",
  "items": [ { "po_item": 10, "verdict": "REDUCE", "verdict_code": "R", "max_qty": 500,
    "reason_code": "COVER_ABOVE_NORM", "reason_text": "Stock + pending already cover 75% of next month's
plan",
    "rule_id": "COVER-01",
    "metrics": { "plan_qty": 2000, "stock_qty": 1140, "open_po_qty": 360, "headroom_qty": 500,
                 "cover_days": 41, "sell_through_pct": 62.5 } } ] }
```

07 Governance outcomes (sent as events)

What happens after a verdict is sent back through `POST /events` with `event_type = GOV_OUTCOME`, so it travels through the same guaranteed queue. This is the audit trail, and it is the data the AI layer will learn from.

FIELD	TYPE	DESCRIPTION
decision_id *	string	From the /decisions response
po_number * / po_item		
outcome *	enum	BUYER_ACCEPTED_REDUCTION · EXCEPTION_REQUESTED · EXCEPTION_APPROVED · EXCEPTION_REJECTED · PO_RELEASED · PO_WITHDRAWN · NOT_CHECKED_TIMEOUT
by_user * / by_role		BUYER · CATHEAD · BMHEAD · SYSTEM
at *	date-time	
new_qty	number	When the buyer reduced the quantity
reason_code	string	From /reason-codes (mandatory for exception approvals)
note	string ≤ 500	Approver's comment

```
{ "msg_id": "ECP300-OUT-D-20261001-000457-01", "seq": 45188, "event_type": "GOV_OUTCOME",
  "occurred_at": "2026-10-01T15:05:12+05:30", "sap_user": "CATHEAD03",
  "governance_outcome": { "decision_id": "D-20261001-000457", "po_number": "4500099001", "po_item": 10,
    "outcome": "EXCEPTION_APPROVED", "by_user": "CATHEAD03", "by_role": "CATHEAD",
    "at": "2026-10-01T15:05:12+05:30", "reason_code": "FEST", "note": "Diwali build-up for DC01" } }
```


08 Exception reason codes

GET /reason-codes SAP reads daily (or on demand) to fill the approver's dropdown.


```
{ "version": "2026-10-01", "reason_codes": [
  { "code": "NEWL", "text": "New launch - no sales history", "active": true },
  { "code": "FEST", "text": "Festive / event build-up", "active": true },
  { "code": "VMOQ", "text": "Vendor minimum order quantity", "active": true },
  { "code": "PRCA", "text": "Price advantage / one-time buy", "active": true },
  { "code": "REPL", "text": "Store replenishment gap", "active": true },
  { "code": "OTHR", "text": "Other (note mandatory)", "active": true } ] }
```


The list is maintained by the planning head in the tool. The codes above are a starting proposal to be confirmed.

09 Errors and retries

STATUS	MEANING	/EVENTS: SAP DOES	/DECISIONS: SAP DOES
200	Success	Remove unit from queue	Use verdict (route only in LIVE)
400	Malformed request (invalid JSON, missing envelope fields)	Unit goes to error in SBGRFCMON; fix the program and restart the unit	NOT_CHECKED; log for IT
401	Wrong credentials / IP not allowed	Retry (alert Basis)	NOT_CHECKED; alert Basis
409	request_id reused with a different body	—	NOT_CHECKED; generate a new request_id
413	Batch over 500 events or 5 MB	Split and resend	—
422	Valid, but cannot be decided (e.g. unknown article)	—	NOT_CHECKED, with message in the work item
5xx / timeout	Tool unavailable	Retry automatically (bgRFC), increasing interval	NOT_CHECKED; continue as today


```
{ "error_code": "VALIDATION_FAILED", "message": "purchase_order.items[0].order_qty must be > 0", "trace_id":
"T-7f3a9c" }
```

10 SAP implementation notes (for the ABAP team)

These are sketches to show the intended pattern. The SAP team adapts them to their own standards.

A. Event push: write in the BAdI, send after commit via bgRFC

```
" In ME_PROCESS_PO_CUST~POST / MB_DOCUMENT_BADI~MB_DOCUMENT_UPDATE:
" build the event and register a bgRFC unit - executed only after COMMIT WORK.
DATA(lo_dest) = cl_bgrfc_destination_outbound=>create( 'ZPOGOV_BGRFC' ).   " defined in SBGRFCCONF
DATA(lo_unit) = lo_dest->create_qrfc_unit( ).
lo_unit->add_queue_name_outbound( 'ZPOGOV_EVENTS' ).                       " one queue = strict order
CALL FUNCTION 'Z_POGOV_SEND_EVENTS' IN BACKGROUND UNIT lo_unit
  EXPORTING is_event = ls_event.                                           " msg_id, seq, payload

" Z_POGOV_SEND_EVENTS (RFC-enabled): POST to /events; if status <> 200 raise an
" exception so bgRFC keeps the unit and retries.
```

B. HTTP call with JSON (used by both endpoints)

```
DATA lo_http TYPE REF TO if_http_client.
cl_http_client=>create_by_destination( EXPORTING destination = 'ZPOGOV_API'
                                       IMPORTING client = lo_http EXCEPTIONS OTHERS = 1 ).
cl_http_utility=>set_request_uri( request = lo_http->request uri = '/decisions' ).  " appended to path prefix
lo_http->request->set_method( if_http_request=>co_request_method_post ).
lo_http->request->set_content_type( 'application/json; charset=utf-8' ).
DATA(lv_json) = /ui2/cl_json=>serialize( data = ls_request compress = abap_true
                                         pretty_name = /ui2/cl_json=>pretty_mode-low_case ).
lo_http->request->set_cdata( lv_json ).
lo_http->send( EXPORTING timeout = 3 EXCEPTIONS OTHERS = 1 ).     " 3 s for /decisions, 10 s for /events
IF sy-subrc = 0. lo_http->receive( EXCEPTIONS OTHERS = 1 ). ENDIF.
IF sy-subrc <> 0.  " timeout or network error
  lv_verdict = 'NOT_CHECKED'.
ELSE.
  lo_http->response->get_status( IMPORTING code = lv_code ).
  IF lv_code = 200.
    /ui2/cl_json=>deserialize( EXPORTING json = lo_http->response->get_cdata( )
                                         pretty_name = /ui2/cl_json=>pretty_mode-low_case
                               CHANGING  data = ls_response ).
  ELSE.
    lv_verdict = 'NOT_CHECKED'.
  ENDIF.
ENDIF.
lo_http->close( ).
```

C. Depth plan bypass after a REDUCE

- On a REDUCE verdict, store `decision_id`, `max_qty` and the date on the PO item (custom fields or a Z-table keyed by PO and item).
- In the existing depth plan check, skip the item only if: the item still carries that decision ID, the new quantity is not more than `max_qty`, and it is the same PO, item and day. Otherwise run the depth check as normal.

- Resubmit with `depth_check = BYPASSED` and `previous_decision_id`; log every bypass.
- The same applies when an approver approves a lower quantity on an exception.

D. Switch and shadow mode

- Parameter `ZPOGOV_ACTIVE` per division (e.g. in TVARVC or a Z-table): blank = skip the step entirely; `S` = call in SHADOW; `L` = call in LIVE and route.
- Store `decision_id`, verdict and summary_text in the workflow container and as a PO header text for visibility.

E. Automatic fallback when the server is down

- Per PO: timeout (3 s), any non-200 answer, or `data_delayed = true` → continue on the previous process and record NOT_CHECKED.
- Health-check job (every minute) calls `/ping`. After 3 failures in a row it sets `ZPOGOV_ACTIVE` to blank for all divisions (previous process for every PO) and alerts IT and planning.
- Switch-back is manual, by an authorised user, once `/ping` is healthy and the bgRFC queue has caught up.
- While the server is down, bgRFC keeps all event units and delivers them in order on recovery.

11 Test checklist (SAP QA ↔ tool test copy)

#	TEST	EXPECTED
1	SM59 connection test	200 from /ping
2	Wrong password	401; nothing stored
3	Create, change, release a PO	3 events accepted in order; the tool shows the PO with the correct open qty
4	Post GR 101, then reverse 102	Open qty goes down, then back up
5	Send the same batch twice	Second time: all in `duplicates`, no double counting
6	Stop the tool for 10 minutes, post documents, restart	Units retried; all events arrive in order; no gaps
7	Event with a missing article	Returned in `parked`; queue continues; IT alerted
8	Decision in SHADOW mode	Verdict stored; PO routed as today
9	Decision in LIVE mode, one case per verdict	Routing as in section 06
10	Tool delayed by more than 3 s	PO continues; NOT_CHECKED recorded
11	Switch off (`ZPOGOV_ACTIVE` blank)	Step skipped; old flow
12	Exception approved with reason code	GOV_OUTCOME event stored against the decision_id

#	TEST	EXPECTED
13	Load: 50 decisions in parallel	95% answered in under 1 s
14	REDUCE, buyer reduces to max_qty and resubmits	Depth check bypassed for that item; tool returns ALLOW; bypass logged
15	Buyer changes the quantity to more than max_qty	Depth check runs as normal; no bypass
16	Stop the server for 5 minutes	After 3 failed health checks the step switches off automatically; POs run on the previous process; alert sent
17	Restart the server; switch back on manually	Queued events delivered in order; feed delay back under the limit; step resumes

The OpenAPI file can be imported into Postman, Insomnia or SAP API tooling to generate test calls with the examples above.

12 Tool-side build status

The PO Governance server running today does **not yet** have these endpoints. Today it serves the planners' screens over plain HTTP on port 8081, and its calculations run in the browser. To receive SAP calls, the following must be added on our side:

- HTTPS listener on 8443 with the company certificate, Basic auth accounts and IP allow-list
- `/ping`, `/reason-codes`, and `/events` (store, de-duplicate, park, apply to open PO / stock figures)
- `/decisions`: the Tool 1 / Tool 2 rules moved from the browser to the server, plus a decision log
- A monitoring page (feed lag, parked events, NOT_CHECKED count) and a shadow-mode comparison report

Once this specification is agreed with the SAP team, these endpoints are built on the server and connected to SAP QA for testing.

Specification v1.4, 1 Oct 2026 (v1.4: SAP figures removed from /decisions, depth_check and depth bypass, data_delayed, automatic switch-off; v1.3: company level, vendor POs only, no transfers, plan in metrics, routing by SAP workflow; v1.1: Stage 2 labels, cut-off rule, open PO split by due month; v1.2: in-server sync of /events and /decisions, Stage 3 note). Changes after sign-off are versioned; any breaking change becomes /v2.